DATABASE MANAGEMENT SYSTEM

Basic SQL Queries & Aggregate Functions

WHERE, ORDER BY, GROUP BY, HAVING and aggregate functions on the online_shop database

Submitted By	
Name	Palle Sai Rithesh
Roll Number	2511021240013
Course	BCA Game Development
Subject	Database Management System
Assignment	Assignment 8 — Basic SQL Queries & Aggregate Functions
Project	Online Shopping System

Assignment 8 — requirements

- Write SQL queries to retrieve and manipulate data from your database.
- Include queries demonstrating: filtering with WHERE, sorting with ORDER BY, grouping with GROUP BY, filtering grouped results with HAVING, and the aggregate functions COUNT(), SUM(), AVG(), MIN() and MAX().
- Provide the SQL commands along with their corresponding outputs and brief explanations.

01 OVERVIEW

Database Used and Query Conventions

All outputs are produced from the sample data inserted in Assignment 7.

The queries run against the **online_shop** database built in Assignments 5 to 7. The table below shows how many rows each table holds.

Table	customers	pincodes	addresses	categories	products
Rows	12	10	14	6	20

Table	orders	order_items	payments	shipments	reviews
Rows	15	30	16	12	15

Clause / function	What it does	Used in
WHERE	Keeps only the rows that satisfy a condition, before any grouping	1.1 to 1.5, 3.2, 5.2, 5.4
ORDER BY	Sorts the result (ASC is the default, DESC reverses it)	2.1, 2.2 and most later queries
GROUP BY	Collapses rows with the same value into one result row	4.1 to 4.4
HAVING	Keeps only the groups that satisfy a condition on an aggregate	5.1 to 5.4, 6.1
COUNT, SUM, AVG, MIN, MAX	Aggregate functions that summarise many rows into one value	3.1 to 3.4 and all grouped queries

> **Note:** The SQL is standard and runs unchanged on MySQL 8. Amounts are shown with two decimals as MySQL shows DECIMAL(10,2) columns. Each query is followed by its output and a short explanation.

02 TASK 1

Filtering Records with WHERE

WHERE keeps only the rows that match a condition.

1.1 Products priced above ₹2,000

Filtering rows with a simple comparison in WHERE.

```
SELECT product_id, name, price
FROM products
WHERE price > 2000
ORDER BY product_id;
```

OUTPUT

product_id	name	price
3	Smart Fitness Band	2499.00
4	27-inch IPS Monitor	14999.00
11	Non-Stick Cookware Set	2299.00
13	Mixer Grinder 750 W	3499.00
15	Adjustable Dumbbell Pair	3299.00
16	Running Shoes	2799.00
17	Wireless Game Controller	3299.00
18	Mechanical Gaming Keyboard	3999.00

(8 rows)

Explanation: Eight of the 20 products cost more than ₹2,000. The monitor is the most expensive at ₹14,999.00. The test is strictly greater than, so a product priced exactly ₹2,000 would not appear.

1.2 Completed orders placed in September 2026

WHERE with AND and BETWEEN on a DATETIME column.

```
SELECT order_id, customer_id, order_date, total_amount
FROM orders
WHERE status = 'Completed'
  AND order_date BETWEEN '2026-09-01 00:00:00' AND '2026-09-30 23:59:59';
```

OUTPUT

order_id	customer_id	order_date	total_amount
6	1	2026-09-02 11:30:00	2598.00
7	6	2026-09-06 16:45:00	2448.00
9	8	2026-09-14 19:35:00	3797.00
10	2	2026-09-20 13:15:00	5497.00

(4 rows)

Explanation: Four completed orders fall in September 2026. BETWEEN includes both end points, and the time part is written out so the whole of 30 September is covered. Order 8 (10 September) is left out because it was cancelled.

1.3 Products whose name contains ‘Wireless’

Pattern matching with LIKE.

```
SELECT product_id, name, price
FROM products
WHERE name LIKE '%Wireless%';
```

OUTPUT

product_id	name	price
1	Wireless Bluetooth Earbuds	1499.00
17	Wireless Game Controller	3299.00

(2 rows)

Explanation: The % sign stands for any run of characters, so LIKE '%Wireless%' finds the word anywhere in the name. Two products match.

1.4 Customers with no phone number

IS NULL tests for a missing value.

```
SELECT customer_id, name, email
FROM customers
WHERE phone IS NULL;
```

OUTPUT

customer_id	name	email
12	Lakshmi Venkatesh	lakshmi.venkatesh@example.com

(1 row)

Explanation: NULL means “unknown”, so *phone = NULL* is never true. IS NULL is the correct test. Only Lakshmi Venkatesh has no phone number stored.

1.5 Products from the Gaming or Sports & Fitness categories

IN matches a value against a list.

```
SELECT product_id, category_id, name
FROM products
WHERE category_id IN (5, 6)
ORDER BY category_id, product_id;
```

OUTPUT

product_id	category_id	name
14	5	Yoga Mat 6 mm
15	5	Adjustable Dumbbell Pair
16	5	Running Shoes
17	6	Wireless Game Controller
18	6	Mechanical Gaming Keyboard
19	6	Gaming Mouse
20	6	Large Mouse Pad

(7 rows)

Explanation: IN (5, 6) is shorthand for category_id = 5 OR category_id = 6. Seven products are returned: three from Sports & Fitness and four from Gaming.

03 TASK 2

Sorting Records with ORDER BY

ORDER BY controls the order in which rows are returned.

2.1 Five most expensive products

ORDER BY … DESC sorts from highest to lowest; LIMIT keeps the first rows.

```
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 5;
```

OUTPUT

name	price
27-inch IPS Monitor	14999.00
Mechanical Gaming Keyboard	3999.00
Mixer Grinder 750 W	3499.00
Adjustable Dumbbell Pair	3299.00
Wireless Game Controller	3299.00

(5 rows)

Explanation: DESC puts the largest value first and LIMIT 5 keeps the first five rows. Without LIMIT all 20 products would be listed.

2.2 Orders sorted by status, then by value

ORDER BY with two columns: the second column breaks ties in the first.

```
SELECT order_id, status, total_amount
FROM orders
ORDER BY status ASC, total_amount DESC;
```

OUTPUT

order_id	status	total_amount
8	Cancelled	3299.00
2	Completed	17098.00
5	Completed	7298.00
10	Completed	5497.00
9	Completed	3797.00
4	Completed	3198.00
3	Completed	2796.00
6	Completed	2598.00
7	Completed	2448.00
1	Completed	2098.00
11	Confirmed	4398.00
12	Confirmed	4149.00
14	Placed	18998.00
15	Placed	4997.00
13	Placed	3197.00

(15 rows)

Explanation: Rows are sorted by status first (Cancelled, Completed, Confirmed, Placed). Inside each status the largest order comes first because of DESC.

04 TASK 3

Aggregate Functions

COUNT, SUM, AVG, MIN and MAX turn many rows into one summary value.

3.1 Counting rows and non-NULL values

COUNT(*) counts rows; COUNT(column) skips NULLs.

```
SELECT COUNT(*) AS total_customers, COUNT(phone) AS customers_with_phone
FROM customers;
```

OUTPUT

total_customers	customers_with_phone
12	11

(1 row)

Explanation: COUNT(*) counts every row, so all 12 customers are counted. COUNT(phone) ignores NULL values and returns 11.

3.2 Revenue from completed orders

SUM adds up a numeric column.

```
SELECT SUM(total_amount) AS completed_revenue
FROM orders
WHERE status = 'Completed';
```

OUTPUT

completed_revenue
46828.00

(1 row)

Explanation: The nine completed orders add up to ₹46,828.00. Cancelled and in-progress orders are excluded by the WHERE clause.

3.3 Average, lowest and highest product price

AVG, MIN and MAX in a single query.

```
SELECT ROUND(AVG(price), 2) AS avg_price, MIN(price) AS min_price, MAX(price) AS max_price
FROM products;
```

OUTPUT

avg_price	min_price	max_price
2499.10	399.00	14999.00

(1 row)

Explanation: The average catalogue price is ₹2,499.10. Prices range from ₹399.00 (Large Mouse Pad) to ₹14,999.00 (monitor).

3.4 Summary of all order lines

All five aggregate functions over order_items.

```
SELECT COUNT(*) AS line_count, SUM(quantity) AS units_sold,
       ROUND(AVG(unit_price), 2) AS avg_unit_price,
       MIN(unit_price) AS min_unit_price, MAX(unit_price) AS max_unit_price
FROM order_items;
```

OUTPUT

line_count	units_sold	avg_unit_price	min_unit_price	max_unit_price
30	36	2719.07	499.00	15499.00

(1 row)

Explanation: The 15 orders contain 30 order lines covering 36 units. The highest unit price, ₹15,499.00, is above any current catalogue price because order 2 bought the monitor before its price fell. This is why unit_price is stored in order_items.

05 TASK 4

Grouping Records with GROUP BY

GROUP BY produces one summary row per group.

4.1 Products per category

GROUP BY produces one result row per category.

```
SELECT category_id, COUNT(*) AS product_count, ROUND(AVG(price), 2) AS avg_price
FROM products
GROUP BY category_id
ORDER BY category_id;
```

OUTPUT

category_id	product_count	avg_price
1	4	5224.00
2	3	1199.00
3	3	899.67
4	3	2232.33
5	3	2265.67
6	4	2324.00

(6 rows)

Explanation: Electronics and Gaming hold four products each; the other four categories hold three. Electronics has the highest average price (₹5,224.00) because of the monitor.

4.2 Orders and value by status

Grouping on a text column with COUNT and SUM.

```
SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_value
FROM orders
GROUP BY status
ORDER BY order_count DESC;
```

OUTPUT

status	order_count	total_value
Completed	9	46828.00
Placed	3	27192.00
Confirmed	2	8547.00
Cancelled	1	3299.00

(4 rows)

Explanation: Nine of the 15 orders are completed, worth ₹46,828.00 in total. Three orders are still placed, two are confirmed and one is cancelled.

4.3 Payments by method

Group payments and total the amount for each method.

```
SELECT payment_method, COUNT(*) AS payment_count, SUM(amount) AS total_amount
FROM payments
GROUP BY payment_method
ORDER BY total_amount DESC;
```

OUTPUT

payment_method	payment_count	total_amount
Card	5	51589.00
UPI	7	39435.00
Net Banking	2	8195.00
COD	2	5645.00

(4 rows)

Explanation: Card is highest at ₹51,589.00, but the figure includes a failed attempt of ₹18,998.00 because no WHERE filter was used. Query 5.4 repeats the idea for successful payments only.

4.4 Shipments per carrier

GROUP BY with two ORDER BY keys.

```
SELECT carrier, COUNT(*) AS shipment_count
FROM shipments
GROUP BY carrier
ORDER BY shipment_count DESC, carrier;
```

OUTPUT

carrier	shipment_count
Delhivery	5
Blue Dart	3
Ecom Express	3
India Post	1

(4 rows)

Explanation: Delhivery handled five of the twelve shipments. Rows with the same count are ordered by carrier name.

06 TASK 5

Filtering Groups with HAVING

HAVING is to groups what WHERE is to rows.

5.1 Categories that hold more than three products

HAVING filters groups after they are formed.

```
SELECT category_id, COUNT(*) AS product_count
FROM products
GROUP BY category_id
HAVING COUNT(*) > 3;
```

OUTPUT

category_id	product_count
1	4
6	4

(2 rows)

Explanation: WHERE cannot use COUNT(*) because the count only exists after grouping, so HAVING is used. Electronics (1) and Gaming (6) have four products each.

5.2 Customers with at least two non-cancelled orders

WHERE removes rows before grouping; HAVING removes groups afterwards.

```
SELECT customer_id, COUNT(*) AS order_count, SUM(total_amount) AS total_spend
FROM orders
WHERE status <> 'Cancelled'
GROUP BY customer_id
HAVING COUNT(*) >= 2
ORDER BY total_spend DESC;
```

OUTPUT

customer_id	order_count	total_spend
5	2	26296.00
2	2	22595.00
3	2	5993.00
1	2	4696.00

(4 rows)

Explanation: WHERE first removes the cancelled order, then the rows are grouped, then HAVING keeps customers with two or more orders. Customer 5 spent the most (₹26,296.00) because of the monitor order.

5.3 Products with an average rating of 4.5 or more

HAVING on an aggregate (AVG) that WHERE cannot use.

```
SELECT product_id, COUNT(*) AS review_count, ROUND(AVG(rating), 2) AS avg_rating
FROM reviews
GROUP BY product_id
HAVING AVG(rating) >= 4.5
ORDER BY avg_rating DESC, product_id;
```

OUTPUT

product_id	review_count	avg_rating
4	1	5.00
9	1	5.00
11	1	5.00
18	1	5.00
8	2	4.50

(5 rows)

Explanation: Five products average 4.5 or better. Four of them have only one review, which is why review_count is shown next to the average: a perfect score from a single review is weak evidence.

5.4 Payment methods that brought in more than ₹10,000

Only successful payments are grouped, then small totals are dropped.

```
SELECT payment_method, SUM(amount) AS paid_amount
FROM payments
WHERE payment_status = 'Paid'
GROUP BY payment_method
HAVING SUM(amount) > 10000
ORDER BY paid_amount DESC;
```

OUTPUT

payment_method	paid_amount
UPI	36136.00
Card	32591.00

(2 rows)

Explanation: Only successful payments are counted. UPI (₹36,136.00) and Card (₹32,591.00) pass the ₹10,000 threshold; Net Banking and COD stay below it.

07 TASK 6

Combining All Clauses

One statement that uses every clause in the correct order.

6.1 Stock summary by category

WHERE, GROUP BY, HAVING, ORDER BY and four aggregates in one statement.

```
SELECT category_id, COUNT(*) AS product_count,
       MIN(price) AS min_price, MAX(price) AS max_price,
       SUM(stock_qty) AS units_in_stock
FROM products
WHERE price > 500
GROUP BY category_id
HAVING COUNT(*) >= 2
ORDER BY units_in_stock DESC, category_id;
```

OUTPUT

category_id	product_count	min_price	max_price	units_in_stock
1	4	1499.00	14999.00	290
3	3	599.00	1250.00	275
2	2	1299.00	1799.00	240
5	3	699.00	3299.00	240
4	3	899.00	3499.00	200
6	3	1599.00	3999.00	163

(6 rows)

Explanation: WHERE first removes products costing ₹500 or less (the T-shirt and the mouse pad). The rest are grouped, HAVING drops groups with fewer than two products, and the result is sorted by stock. Electronics holds the most units (290).

08 TASK 7

Manipulating Data with UPDATE, DELETE and INSERT

Each change runs inside a transaction and is rolled back, so the data stays exactly as inserted in Assignment 7.

7.1 Give every Clothing product a 10% discount

UPDATE changes existing rows. The condition in WHERE decides which rows are touched.

```
START TRANSACTION;
UPDATE products SET price = ROUND(price * 0.90, 2) WHERE category_id = 2;
SELECT product_id, name, price FROM products WHERE category_id = 2;
```

OUTPUT

```
Query OK, 3 rows affected
Rows matched: 3  Changed: 3  Warnings: 0
```

product_id	name	price
5	Cotton Round-Neck T-Shirt	449.10
6	Slim Fit Denim Jeans	1169.10
7	Hooded Sweatshirt	1619.10

(3 rows)

```
ROLLBACK;
SELECT product_id, name, price FROM products WHERE category_id = 2;
```

OUTPUT

product_id	name	price
5	Cotton Round-Neck T-Shirt	499.00
6	Slim Fit Denim Jeans	1299.00
7	Hooded Sweatshirt	1799.00

(3 rows)

Explanation: Three rows were changed and the new prices (for example ₹499.00 became ₹449.10) were visible inside the transaction. ROLLBACK undid the change, so the original prices are back.

7.2 Remove very low-rated reviews

```
START TRANSACTION;
DELETE FROM reviews WHERE rating <= 2;
SELECT COUNT(*) AS remaining_reviews FROM reviews;
```

OUTPUT

```
Query OK, 1 row affected
```

remaining_reviews
14

(1 row)

```
ROLLBACK;
SELECT COUNT(*) AS remaining_reviews FROM reviews;
```

OUTPUT

remaining_reviews
15

(1 row)

Explanation: Only one review has a rating of 2 or less, so one row was deleted (15 became 14). After ROLLBACK the count is 15 again. A DELETE without WHERE would remove every row, which is why the condition matters.

7.3 Register a new customer

```
START TRANSACTION;
INSERT INTO customers (name, email, phone)
VALUES ('Nisha Rao', 'nisha.rao@example.com', '9876500013');
SELECT customer_id, name, email FROM customers WHERE email = 'nisha.rao@example.com';
ROLLBACK;
```

OUTPUT

```
Query OK, 1 row affected
```

customer_id	name	email
13	Nisha Rao	nisha.rao@example.com

(1 row)

Explanation: customer_id was not supplied; AUTO_INCREMENT assigned the next number (13). The row is then rolled back.

09 SUMMARY

What These Queries Show

- **WHERE** filters rows, **HAVING** filters groups. WHERE runs first, so it cannot see aggregate results.
- **GROUP BY** with an aggregate gives one summary row per group; every non-aggregated column in SELECT must appear in GROUP BY.
- Aggregates ignore NULL values (COUNT(phone) is 11 while COUNT(*) is 12).
- The logical order of a query is FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT.
- Data-changing statements can be tested safely inside a transaction and undone with ROLLBACK.